

# MAFIAGAME 프로젝트 입문 강의서

## 현재 코드 기준 구조와 데이터 흐름

이 문서는 Spring Boot를 처음 보는 사람이 현재 프로젝트의 실행 구조를 따라가며 학습할 수 있도록 만든 강의형 안내서입니다. 로그인과 회원가입부터 방 목록, 방 생성, 대기실, 실시간 채팅과 참가자 준비 상태까지 한 사용자의 행동이 어떤 코드와 데이터베이스를 거치는지 순서대로 설명합니다.

핵심 결론은 간단합니다. 이 프로젝트는 브라우저가 요청을 보내면 Controller가 입구가 되고, Service가 규칙을 판단하며, MyBatis Mapper가 데이터베이스를 읽고 쓰는 MVC 구조를 기본으로 합니다. 대기실에서 계속 바뀌는 참가자와 준비 상태는 DB가 아니라 서버 메모리와 WebSocket으로 전달됩니다.

분석 기준: C:\workspace-sts-5.3.0\mafiagame | 기준일: 2026년 9월 16일

| 대상 독자              | 이 문서에서 얻는 것                                          |
|--------------------|------------------------------------------------------|
| Java와 웹 개발이 처음인 사람 | 프로젝트를 실행하고 요청이 서버를 통과하는 길을 이해합니다.                    |
| Spring MVC를 배우는 사람 | Controller, Service, Mapper, Domain, DTO의 역할을 구분합니다. |
| 현재 프로젝트를 이어받는 개발자  | 구현된 범위와 아직 게임 로직으로 확장해야 할 경계를 빠르게 파악합니다.             |

읽는 순서: 1장부터 4장까지는 웹과 Spring의 기초, 5장부터 8장까지는 이 프로젝트의 실제 흐름, 9장 이후는 파일 지도와 다음 학습 과제입니다.

# 1. 먼저 알아야 할 전체 그림

코드를 처음 볼 때 모든 파일을 한 줄씩 읽으려고 하면 오히려 구조가 흐려집니다. 먼저 사용자의 행동을 하나의 긴 흐름으로 보고, 그다음 각 단계의 담당 파일을 찾는 방식이 가장 빠릅니다.

## 1.1 사용자가 보는 서비스 흐름

| 브라우저 화면 | HTTP Controller | Service 규칙 | MyBatis Mapper | MariaDB |
|---------|-----------------|------------|----------------|---------|
|---------|-----------------|------------|----------------|---------|

브라우저 -> 컨트롤러 -> 서비스 -> 매퍼 -> DB

- **회원가입:** 회원가입 화면에서 입력한 값이 /signup POST 요청으로 서버에 들어갑니다.
- **로그인:** Spring Security가 로그인 요청을 가로채고 이메일로 사용자를 찾은 뒤 세션을 만듭니다.
- **로비:** RoomController가 RoomService를 호출하고, RoomMapper의 SQL이 방 목록과 인원 수를 조회합니다.
- **대기실:** 초기 화면은 Thymeleaf가 만들고, 이후 참가자와 준비 상태는 WebSocket/STOMP 메시지가 갱신합니다.
- **채팅:** 브라우저가 /ws WebSocket으로 연결한 뒤 STOMP SEND를 보내면 서버가 같은 방의 topic으로 다시 방송합니다.

## 1.2 폴더 구조를 읽는 법

|                                    |                        |
|------------------------------------|------------------------|
| src/main/java/kr/or/oti/mafiagame/ |                        |
| MafiagameApplication.java          | 애플리케이션 시작점             |
| config/                            | Security와 WebSocket 설정 |
| controller/                        | HTTP와 WebSocket 메시지 입구 |
| service/                           | 검증, 업무 규칙, 실시간 상태 관리   |
| dao/                               | MyBatis Mapper 인터페이스   |
| domain/                            | DB와 가까운 객체             |
| dto/                               | 메시지와 화면 전달용 객체         |
| security/                          | 로그인 사용자 표현             |
| exception/                         | WebSocket 오류 타입        |
| src/main/resources/                |                        |
| templates/                         | Thymeleaf HTML         |
| static/css/                        | 화면 스타일                 |
| static/js/                         | 브라우저 동작과 WebSocket 코드  |
| mappers/                           | 실제 SQL이 들어 있는 XML      |
| application.properties             | DB와 MyBatis 설정         |

README.md의 예시 구조 설명은 현재 실제 폴더 이름과 일부 다릅니다. 현재 소스에서 기준이 되는 구조는 controller, service, dao, domain, dto, config, security입니다. 학습할 때는 문서의 설명보다 실제 파일 트리를 우선 확인하세요.

### 1.3 이 프로젝트의 구현 범위

| 현재 구현됨                        | 아직 구현되지 않음                    |
|-------------------------------|-------------------------------|
| 회원가입, BCrypt 비밀번호 저장, 이메일 로그인 | 방 입장 시 비밀번호 검증과 DB 참가 등록      |
| DB 기반 방 목록과 방 생성              | 게임 시작, 직업 배정, 낮과 밤, 투표, 승패 판정 |
| WebSocket 채팅과 실시간 접속자/준비 상태   | 게임 진행용 WebSocket 메시지와 페이즈 권한  |
| Thymeleaf 화면과 기본 반응형 CSS      | 실제 사용자 전적, 친구, 업적 데이터 연동      |

## 2. Spring Boot가 애플리케이션을 시작하는 과정

Spring Boot 애플리케이션은 main 메서드에서 시작하지만, 실제로 많은 일을 Spring이 대신 준비합니다.

@SpringBootApplication이 붙은 클래스가 기준 패키지를 정하고, Spring은 그 아래의 @Controller, @Service, @Configuration, @Mapper를 찾아 객체를 생성합니다.

### 2.1 시작점

파일: src/main/java/kr/or/oti/mafiagame/MafiagameApplication.java

```
@SpringBootApplication
@MapperScan("kr.or.oti.mafiagame")
public class MafiagameApplication {
    public static void main(String[] args) {
        SpringApplication.run(MafiagameApplication.class, args);
    }
}
```

- **@SpringBootApplication**: 자동 설정, 컴포넌트 스캔, 설정 클래스 검색을 한 번에 활성화합니다.
- **@MapperScan**: MyBatis Mapper 인터페이스를 Spring Bean처럼 주입할 수 있게 검색합니다.
- **SpringApplication.run**: 내장 서버, MVC, Security, MyBatis, WebSocket에 필요한 객체를 조립합니다.

### 2.2 의존성은 무엇을 제공하는가

| build.gradle 의존성              | 프로젝트에서 사용하는 기능                     |
|-------------------------------|------------------------------------|
| spring-boot-starter-web       | Spring MVC, 내장 서버, HTTP 요청 처리      |
| spring-boot-starter-thymeleaf | 서버에서 HTML에 데이터를 넣어 렌더링             |
| spring-boot-starter-security  | 로그인, 세션, 접근 권한, 비밀번호 인증            |
| mybatis-spring-boot-starter   | Java Mapper와 XML SQL 연결            |
| spring-boot-starter-websocket | WebSocket과 STOMP 메시지 브로커           |
| mariadb-java-client           | MariaDB JDBC 연결                    |
| lombok                        | Getter, Setter, Builder 등 반복 코드 생성 |

### 2.3 application.properties 읽기

파일: src/main/resources/application.properties

```
spring.datasource.url=jdbc:mariadb://localhost:23306/mafiaweb
spring.datasource.username=${DB_USERNAME:root}
```

```
spring.datasource.password=${DB_PASSWORD:}  
mybatis.mapper-locations=classpath:/mappers/**/*.xml  
mybatis.configuration.map-underscore-to-camel-case=true
```

`${DB_USERNAME:root}`처럼 쓰인 표현은 환경 변수 `DB_USERNAME`이 있으면 그 값을 쓰고, 없으면 `root`를 사용한다는 의미입니다. 비밀번호는 기본값이 빈 값입니다. 실제 개발 환경에서는 계정 정보를 소스에 직접 적기보다 환경 변수나 별도 비밀 저장소를 사용합니다.

`map-underscore-to-camel-case=true`는 SQL 결과의 `current_players`를 Java 프로퍼티 `currentPlayers`로 연결하는 데 도움을 줍니다. 다만 `User` 클래스의 `user_level`처럼 Java 필드 자체가 언더스코어 이름이면, 다른 필드와 명명 방식이 섞이므로 유지보수 때 주의해야 합니다.

### 3. MVC를 실제 코드로 이해하기

MVC는 파일을 세 종류로 나누는 규칙이 아니라, 책임을 나누는 사고방식입니다. 사용자의 요청을 받는 곳, 규칙을 판단하는 곳, 데이터를 저장하는 곳을 분리하면 코드가 커져도 어느 부분을 수정해야 하는지 찾기 쉬워집니다.

| 역할         | 이 프로젝트의 위치                           | 쉽게 말하면                                 |
|------------|--------------------------------------|----------------------------------------|
| Controller | controller/*.java                    | 주소와 메시지의 입구. 화면 이름이나 응답을 결정합니다.        |
| Service    | service/*.java                       | 회원가입 검증, 방 생성, 실시간 상태 같은 업무 규칙을 수행합니다. |
| Mapper     | dao/*.java + resources/mappers/*.xml | Java 메서드와 SQL을 연결합니다.                  |
| Domain     | domain/*.java                        | DB에서 읽고 쓰는 데이터 구조를 표현합니다.              |
| DTO        | dto/*.java                           | 채팅, 준비 상태 등 특정 메시지의 입력과 출력 모양을 정의합니다.  |
| Template   | templates/**/*.html                  | 서버가 전달한 Model을 화면으로 바꿉니다.              |

#### 3.1 Controller는 얇게 유지된다

파일: `controller/RoomController.java`

```
@GetMapping("/{", "/rooms"})
public String roomList(Model model) {
    model.addAttribute("rooms", roomService.getRooms());
    return "rooms/list";
}
```

위 메서드가 하는 일은 세 가지입니다. URL을 매핑하고, Service를 호출하고, 결과를 Model에 넣은 뒤 `rooms/list`라는 Thymeleaf 템플릿을 반환합니다. SQL이나 비밀번호 규칙을 Controller 안에 넣지 않는 것이 이 구조의 중요한 약속입니다.

### 3.2 Service는 규칙을 모으는 곳이다

RoomService.createRoom은 제목 길이, 인원 수, 비밀번호 길이를 확인하고, 비밀번호를 BCrypt로 해시한 Room 객체를 만든 다음 두 개의 INSERT를 하나의 트랜잭션 안에서 실행합니다. SignupService.signup도 입력 정규화와 검증, 이메일 중복 확인, 비밀번호 해시, 사용자 INSERT를 담당합니다.

### 3.3 Mapper는 SQL을 습니다

Controller와 Service는 SELECT 문을 직접 알지 않습니다. RoomMapper.findAll 같은 메서드를 호출하고, 실제 SQL은 RoomMapper.xml에 있습니다. 그래서 DB 컬럼이나 JOIN을 바꿀 때는 Mapper XML과 결과 객체의 연결을 함께 확인하면 됩니다.

| 호출                      | SQL                                        | 결과             |
|-------------------------|--------------------------------------------|----------------|
| UserMapper.findByEmail  | email로 user 조회                             | Optional<User> |
| RoomMapper.findAll      | game_room과 user, room_members JOIN 및 COUNT | List<RoomList> |
| RoomMapper.findById     | 특정 방 한 개와 현재 DB 인원 수 조회                    | RoomList       |
| RoomMapper.insert       | game_room INSERT 및 생성된 roomId 수신           | Room           |
| RoomMapper.insertMember | 방장과 방의 관계 INSERT                           | 영향받은 행 수       |

## 4. 회원가입과 로그인 흐름

인증은 두 단계를 구분해서 이해해야 합니다. 회원가입은 사용자를 DB에 만드는 과정이고, 로그인은 이미 있는 사용자를 찾아 현재 브라우저를 인증된 상태로 바꾸는 과정입니다.

### 4.1 회원가입 화면에서 DB까지

|             |                |               |            |          |
|-------------|----------------|---------------|------------|----------|
| signup.html | AuthController | SignupService | UserMapper | user 테이블 |
|-------------|----------------|---------------|------------|----------|

브라우저 -> 컨트롤러 -> 서비스 -> 매퍼 -> DB

파일: `controller/AuthController.java`

```
POST /signup
fields: nickname, email, password, passwordConfirm, agreement
성공: redirect:/login?signup
실패: auth/signup + signupError + 입력값 일부 복원
```

- **정규화:** 닉네임과 이메일의 앞뒤 공백을 제거하고 이메일은 소문자로 바꿉니다.
- **검증:** 닉네임 2~30자, 이메일 형식, 비밀번호 8자 이상 및 UTF-8 72바이트 이하, 확인 값 일치, 약관 동의를 확인합니다.
- **중복 방지:** `UserMapper.existsByEmail`로 먼저 확인하고, DB UNIQUE 제약 위반도 `DataIntegrityViolationException`으로 다시 처리합니다.
- **보안 저장:** 원문 비밀번호를 저장하지 않고 `PasswordEncoder`, 즉 `BCryptPasswordEncoder`로 해시합니다.

### 4.2 로그인은 Spring Security가 가로챈다

화면: `templates/auth/login.html` | 설정: `config/SecurityConfig.java`

```
form action=/login method=post
usernameParameter = email
passwordParameter = password
성공 후 이동 = /rooms
실패 후 이동 = /login?error
```

로그인 폼은 `AuthController`의 POST 메서드로 가지 않습니다. `SecurityConfig`가 `loginProcessingUrl('/login')`을 등록했기 때문에 Spring Security 필터가 요청을 먼저 처리합니다. 이때 email 필드가 `usernameParameter`로 지정되어 있어, Security가 `UserDetailsService.loadUserByUsername(email)`을 호출합니다.

파일: `service/CustomUserDetailsService.java`

```
String normalizedEmail = email == null ? "" : email.trim().toLowerCase(Locale.ROOT);
return userMapper.findByEmail(normalizedEmail)
    .map(CustomUserDetails::new)
    .orElseThrow(...);
```

`CustomUserDetails`는 Spring Security의 User를 상속합니다. 기본 인증 정보에는 이메일과 해시된 비밀번호가 들어가고, 프로젝트가 화면과 실시간 메시지에서 필요로 하는 `userId`와 `nickname`을 추가로 보관합니다. 그래서

Thymeleaf에서는 principal.nickname, 서버에서는 @AuthenticationPrincipal CustomUserDetails를 사용할 수 있습니다.

## 4.3 접근 권한 규칙

| 경로                            | 현재 권한   | 이유                              |
|-------------------------------|---------|---------------------------------|
| / , /rooms , /login , /signup | 비로그인 허용 | 로비와 인증 화면은 누구나 볼 수 있습니다.        |
| /css/** , /js/** , /error     | 비로그인 허용 | 정적 리소스와 오류 페이지입니다.              |
| /ws/**                        | 로그인 필요  | WebSocket 연결도 인증 사용자를 대상으로 합니다. |
| 그 외 모든 요청                     | 로그인 필요  | 방 생성과 방 상세 등은 인증된 사용자만 접근합니다.   |

## 5. 로비에서 방을 만들기까지

로비는 DB에 저장된 방 목록을 서버에서 HTML로 그립니다. 검색과 상태 필터는 서버 재조회가 아니라 브라우저의 작은 JavaScript가 이미 받은 카드들을 숨기고 보여 주는 방식입니다.

### 5.1 방 목록 조회

실제 호출 흐름

```
GET / 또는 GET /rooms
RoomController.roomList
-> RoomService.getRooms
-> RoomMapper.findAll
-> RoomMapper.xml SELECT
-> RoomList 목록
Model.rooms -> templates/rooms/list.html
```

RoomMapper.findAll은 game\_room을 중심으로 방장 사용자와 room\_members를 JOIN합니다.

COUNT(rm.user\_id)로 DB에 등록된 참가자 수를 세고, 비밀번호가 있으면 locked=true로 계산합니다. 대기 중 방을 먼저 정렬하고, 같은 상태 안에서는 생성 시간이 최근인 방을 위로 올립니다.

### 5.2 방 생성 입력과 서버 검증

파일: controller/RoomController.java

```
POST /rooms
입력: title, maxPlayers, password
인증 사용자: CustomUserDetails user
성공: redirect:/rooms/{roomId}
실패: rooms/create + roomError
```

| 입력         | 서버 규칙             | DB에 저장되는 형태                           |
|------------|-------------------|---------------------------------------|
| title      | 2자 이상 100자 이하     | 앞뒤 공백 제거 후 game_room.title            |
| maxPlayers | 4 이상 8 이하         | game_room.max_players                 |
| password   | 없거나 4~20자         | 있으면 BCrypt 해시, 없으면 NULL               |
| hostUserId | 현재 로그인 사용자 ID     | game_room.host_user_id 및 room_members |
| status     | 서비스가 WAITING으로 지정 | game_room.status                      |

방 생성은 game\_room INSERT 후 방장 자신을 room\_members에 INSERT합니다. 두 작업 중 하나라도 실패하면 @Transactional에 의해 함께 롤백되는 것이 의도입니다. 방장은 DB에 등록되지만, 다른 사용자의 입장 처리 API는 현재 아직 구현되지 않았습니다.

## 5.3 화면과 실제 기능의 차이

- **비밀번호 토글**: create.html의 JavaScript가 비밀번호 필드를 활성화하거나 비활성화합니다. 최종 검증은 RoomService가 합니다.
- **방 입장 버튼**: list.html에서 /rooms/{id}로 이동하지만, RoomController는 현재 방 상세 화면을 보여 주는 단계입니다.
- **관전하기 문구**: status가 PLAYING이면 관전하기라고 표시하지만, 실제 관전 권한이나 관전 데이터 흐름은 아직 없습니다.
- **현재 접속자 128명**: 화면에 고정된 표시값이며 실제 접속자 수를 계산하는 기능이 아닙니다.

## 6. 데이터베이스와 MyBatis 흐름

DB 스키마를 먼저 보면 Java 객체의 의미가 선명해집니다. 현재 스키마는 사용자, 사용자 전적, 게임방, 방 참가 관계를 분리한 관계형 구조입니다.

### 6.1 테이블 관계

| 테이블          | 핵심 컬럼                                             | 현재 사용 방식                           |
|--------------|---------------------------------------------------|------------------------------------|
| user         | user_id, user_name, email, password               | 회원가입과 로그인 사용자 조회                   |
| user_stats   | user_id, total_games, wins, rating                | 스키마는 있으나 현재 Controller는 사용하지 않음    |
| game_room    | room_id, host_user_id, title, max_players, status | 방 생성과 방 목록/상세 조회                   |
| room_members | room_id, user_id, is_ready, role, is_alive        | 방장 등록과 DB 인원 수 계산. 준비/역할 업데이트는 미구현 |

#### 개념 관계

```

user 1 --- N game_room (host_user_id)
user 1 --- N room_members N --- 1 game_room
user 1 --- 1 user_stats
  
```

### 6.2 Domain과 RoomList를 나누는 이유

Room은 방을 INSERT할 때 필요한 값에 가깝습니다. 반면 RoomList는 방 목록에 보여 줄 hostName, currentPlayers, locked까지 포함합니다. 즉 하나는 저장 모델이고 다른 하나는 조회 결과 모델입니다. SQL JOIN과 COUNT의 결과를 Room에 억지로 넣지 않고 RoomList로 분리한 점이 핵심입니다.

| 객체              | 대표 필드                                               | 사용 위치               |
|-----------------|-----------------------------------------------------|---------------------|
| User            | userId, userName, email, password                   | 회원가입 INSERT, 로그인 조회 |
| Room            | hostUserId, title, roomPassword, maxPlayers, status | 방 생성 INSERT         |
| RoomList        | title, hostName, currentPlayers, locked             | 로비와 방 상세 화면         |
| RoomParticipant | userId, nickname, host, ready                       | WebSocket 실시간 상태    |

## 6.3 MyBatis XML을 읽는 순서

- **namespace**:XML이 어떤 Mapper 인터페이스에 연결되는지 확인합니다.
- **id**:Java 인터페이스 메서드 이름과 같은지 확인합니다.
- **resultType** 또는 **resultMap**:SQL 결과를 어떤 Java 객체에 넣는지 확인합니다.
- **#{parameter}**:메서드 파라미터를 안전하게 바인딩하는 자리입니다.
- **useGeneratedKeys**:INSERT 뒤 DB가 만든 AUTO\_INCREMENT 키를 객체에 다시 넣습니다.

UserMapper.xml은 User의 user\_id를 userId, user\_name을 userName으로 명시적으로 매핑합니다.

RoomMapper.xml은 SQL 별칭과 map-underscore-to-camel-case 설정을 함께 사용해 current\_players를 currentPlayers로 연결합니다.

## 7. 방 상세 화면과 두 종류의 상태

방 상세 화면을 이해할 때는 DB 상태와 실시간 접속 상태를 분리해서 보아야 합니다. 화면의 제목, 방장, 최대 인원은 DB에서 읽고, 지금 이 브라우저 세션이 연결되어 있는지와 준비 여부는 WebSocket 서버 메모리에서 관리합니다.

### 7.1 HTTP로 초기 화면을 그리는 과정

초기 페이지 흐름

```
GET /rooms/{roomId}
RoomController.roomDetail
-> roomService.getRoom(roomId)
-> roomService.getMemberNames(roomId)
-> Model(roomId, nickname, userId, room, members)
-> templates/rooms/detail.html
```

detail.html의 body에는 data-room-id, data-nickname, data-user-id, data-capacity가 들어갑니다. chat.js는 이 값을 읽어 어느 방에 연결할지, 현재 사용자를 누구로 표시할지, 빈 자리를 몇 개 만들지 결정합니다.

### 7.2 존재하지 않는 방에 대한 현재 처리

roomDetail에서 roomService.getRoom(roomId)가 null이면 오류 페이지 대신 Moonlight Mafia라는 예시 방과 예시 참가자 목록을 넣어 화면을 보여 줍니다. 이것은 화면 데모에는 도움이 되지만, 같은 roomId로 WebSocket join이나 chat을 시도하면 RoomPresenceService와 ChatController가 존재하지 않는 게임방이라고 거절합니다. 실제 서비스에서는 404 또는 사용자에게 명확한 안내 화면을 반환하는 방향이 더 자연스럽습니다.

### 7.3 RoomPresenceService의 메모리 자료구조

파일: service/RoomPresenceService.java

```
participantsByRoom
roomId -> sessionId -> RoomParticipant
```

```
roomBySession
sessionId -> roomId
```

monitor

두 Map을 동시에 바꿀 때 사용하는 synchronized 기준

participantsByRoom은 같은 방에 연결된 WebSocket 세션들을 모읍니다. roomBySession은 세션이 어느 방에 속해 있는지 빠르게 찾기 위한 역방향 인덱스입니다. 두 Map을 함께 수정해야 하므로 monitor 객체를 기준으로 synchronized 블록을 사용합니다.

## 7.4 참여, 준비, 나가기

| 상황         | 검사와 처리                                        | 방 전체에 보내는 결과                   |
|------------|-----------------------------------------------|--------------------------------|
| join       | 세션과 Principal이 있는지, 방이 DB에 존재하는지 확인. 방장 여부 계산 | /topic/rooms/{roomId}/presence |
| ready      | 세션이 먼저 해당 방에 join했는지 확인.<br>ready 값만 교체       | 새 참가자 목록 전체 방송                 |
| disconnect | SessionDisconnectEvent에서 세션 제거                | 남아 있는 참가자 목록 방송                |

현재 준비 상태는 `room_members.is_ready`에 저장되지 않습니다. 브라우저가 연결된 동안만 살아 있는 서버 메모리 값이며, 서버 재시작이나 다른 서버 인스턴스로 이동하면 사라집니다. 이것은 현재 MVP 단계의 단순한 선택이고, 영속적인 게임 상태가 필요해지면 별도 설계가 필요합니다.

## 8. WebSocket과 STOMP 채팅 흐름

HTTP는 요청을 보내고 응답을 받는 데 적합하지만, 대기실에서 누군가 준비 버튼을 누를 때마다 모든 브라우저에 즉시 알려 주려면 서버가 먼저 메시지를 보낼 수 있어야 합니다. WebSocket은 연결을 오래 유지하고 양방향으로 데이터를 전달하는 통로입니다. STOMP는 그 통로 위에서 목적지와 메시지 형식을 정하는 약속입니다.

### 8.1 서버 설정

파일: `config/WebSocketConfig.java`

```
registry.enableSimpleBroker("/topic", "/queue");
registry.setApplicationDestinationPrefixes("/app");
registry.setUserDestinationPrefix("/user");
registry.addEndpoint("/ws");
```

| 접두사    | 의미                     | 이 프로젝트 예시           |
|--------|------------------------|---------------------|
| /app   | 브라우저가 서버로 보내는 목적지      | /app/rooms/1/chat   |
| /topic | 방 전체가 구독하는 방송 목적지      | /topic/rooms/1/chat |
| /user  | 특정 로그인 사용자에게 보내는 목적지   | /user/queue/errors  |
| /queue | 개인 응답에 자주 쓰는 broker 경로 | errors와 함께 사용       |

### 8.2 브라우저의 연결 순서

파일: `static/js/chat.js`

1. `new WebSocket('ws(s)://현재호스트/ws')`
2. CONNECT frame 전송
3. CONNECTED 수신
4. presence, chat, errors SUBSCRIBE
5. `/app/rooms/{roomId}/join` SEND
6. 사용자가 채팅 또는 준비 상태 SEND
7. beforeunload 때 DISCONNECT

이 프로젝트는 STOMP 전용 JavaScript 라이브러리를 사용하지 않고 `chat.js`에서 frame을 직접 만들고 해석합니다. `createFrame`은 command, headers, body, 널 문자로 하나의 프레임을 만들고, `parseFrame`과 `consumeFrames`는 서버에서 온 여러 프레임을 분리해 처리합니다. 라이브러리 없이 프로토콜을 직접 보여 준다는 교육적 장점이 있지만, 기능이 늘면 표준 클라이언트 라이브러리 사용을 검토할 수 있습니다.

## 8.3 채팅 메시지 계약

| 방향         | 목적지                    | 입력 또는 출력                                                 | 서버 처리                      |
|------------|------------------------|----------------------------------------------------------|----------------------------|
| 브라우저 -> 서버 | /app/rooms/{id}/chat   | ChatMessageRequest<br>{ content }                        | 로그인, 방 존재, 빈 값, 300자<br>검증 |
| 서버 -> 방 전체 | /topic/rooms/{id}/chat | ChatMessage { roomId, type,<br>sender, content, sentAt } | Instant.now()를 붙여 방송       |
| 서버 -> 개인   | /user/queue/errors     | ChatError { type, message }                              | 예외를 해당 사용자에게만 전달           |

ChatController는 principal이 없으면 거절하고, content를 strip한 뒤 비어 있거나 300자를 초과하면 RoomWebSocketException을 던집니다. 출력할 때는 sender와 content를 JSON으로 보내고, 브라우저는 appendMessage에서 textContent를 사용해 DOM에 넣습니다. 이 방식은 받은 문자열을 HTML로 해석하지 않으므로 기본적인 XSS 위험을 줄이는 데 도움이 됩니다.

## 9. 실시간 참가자와 준비 상태의 끝까지 흐름

채팅과 참가자 목록은 같은 WebSocket 연결을 사용하지만, 목적지와 서버 처리 클래스가 다릅니다. 이 차이를 분리해서 읽으면 WebSocket 코드가 훨씬 덜 복잡해집니다.

### 9.1 입장 메시지

실제 처리 순서

브라우저

```
SEND /app/rooms/1/join
body: {}
```

RoomPresenceController.join

```
-> RoomPresenceService.join
  -> DB에서 방 확인
  -> Principal에서 userId, nickname 확인
  -> participantsByRoom에 세션 등록
  -> RoomPresenceState 생성
  -> /topic/rooms/1/presence 방송
```

join은 데이터베이스의 room\_members에 새 행을 넣지 않습니다. 여기서 말하는 참가자는 '현재 WebSocket으로 연결된 브라우저 세션'입니다. 따라서 DB 참가자 수와 실시간 접속자 수가 서로 다를 수 있습니다.

### 9.2 준비 버튼 메시지

실제 처리 순서

브라우저

```
SEND /app/rooms/1/ready
body: { "ready": true }
```

RoomPresenceController.updateReady

```
-> RoomPresenceService.updateReady
  -> sessionId가 roomId에 join했는지 확인
  -> 해당 RoomParticipant의 ready 교체
  -> 전체 참가자 목록 재방송
```

브라우저는 처음부터 준비 버튼을 활성화하지 않습니다. presence 메시지를 받아 현재 사용자 자신의 participant를 찾았을 때 presenceReady=true로 만들고, 연결 상태도 온라인일 때만 버튼을 활성화합니다. 서버가 허용한 상태를 다시 방송하므로 모든 사용자의 화면이 같은 목록으로 수렴합니다.

### 9.3 나가기와 재연결

브라우저가 닫히거나 네트워크가 끊기면 WebSocket의 SessionDisconnectEvent가 발생하고,

RoomPresenceService.handleDisconnect가 leave를 호출합니다. 브라우저 쪽 close 이벤트는 3초 뒤 connect를

다시 시도합니다. beforeunload에서는 shouldReconnect=false로 바꾸어 정상적으로 페이지를 떠날 때 불필요한 재연결을 막습니다.

| 현재 설계의 장점                       | 현재 설계의 한계                        |
|---------------------------------|----------------------------------|
| 한 번의 상태 방송으로 모든 브라우저를 갱신        | 상태가 서버 메모리에만 있어 서버 재시작 시 사라짐     |
| synchronized로 단일 서버 안의 동시 수정 보호 | 여러 서버를 띄우면 인스턴스마다 Map이 달라짐       |
| 세션 ID와 사용자 ID를 함께 식별            | 한 사용자가 여러 탭을 열면 세션별 참가자로 보일 수 있음 |
| 재연결 시 join으로 상태 복원 시도           | 실제 방 입장 권한과 재접속 정책은 아직 없음        |

## 10. 파일별 학습 지도

아래 표는 코드를 읽을 때 어디서부터 시작할지 알려 주는 지도입니다. 한 번에 모든 CSS까지 분석하지 말고, 먼저 Java 흐름과 템플릿에서 사용하는 Model 이름을 맞춰 보세요.

### 10.1 시작과 설정

`MafiagameApplication.java` Spring Boot 시작점과 Mapper 스캔.

`config/SecurityConfig.java` 공개 URL, 로그인 URL, 로그아웃, 인증 필요 경로.

`config/WebSocketConfig.java` STOMP broker와 /ws 엔드포인트.

`application.properties` MariaDB와 MyBatis 연결 설정.

### 10.2 HTTP Controller

`controller/AuthController.java` 로그인/회원가입 화면과 회원가입 POST.

`controller/RoomController.java` 로비, 방 생성, 방 상세 화면.

`controller/UserController.java` 현재는 예시 사용자 프로필 화면.

### 10.3 WebSocket Controller

`controller/ChatController.java` 채팅 입력을 검증하고 방 전체로 방송.

`controller/RoomPresenceController.java` join과 ready 메시지를 PresenceService로 전달.

`exception/RoomWebSocketException.java` WebSocket 처리 중 사용자에게 보여 줄 오류.

### 10.4 Service와 보안

`service/SignupService.java` 회원가입 검증, 중복 확인, BCrypt 저장.

`service/CustomUserDetailsService.java` 로그인 시 이메일로 User를 조회.

`service/RoomService.java` 방 조회와 방 생성 검증/트랜잭션.

`service/RoomPresenceService.java` 메모리 기반 접속자와 준비 상태.

`security/CustomUserDetails.java` Spring Security 사용자에게 userId와 nickname 추가.

### 10.5 화면과 브라우저

`templates/auth/*.html` 로그인과 회원가입 화면.

`templates/rooms/list.html` 로비, 검색, 상태 필터.

`templates/rooms/create.html` 방 생성 입력과 비밀번호 토글.

`templates/rooms/detail.html` 대기실과 채팅 UI, data-\* 초기값.

`static/js/chat.js` STOMP frame, 채팅, 참가자 렌더링, 재연결.

`static/css/*.css` auth, lobby/room, profile 화면 스타일.

## 11. 처음부터 따라가는 실습형 시나리오

이 장은 실제 사용자의 행동을 따라가며 어디에 중단점을 찍고 어떤 값을 확인할지 설명합니다. 코드를 학습할 때는 기능을 읽는 것보다 한 시나리오의 입력과 출력이 어떻게 바뀌는지 추적하는 것이 효과적입니다.

### 11.1 회원가입

- 브라우저에서 /signup을 열고 닉네임, 이메일, 비밀번호, 약관을 입력합니다.
- POST /signup이 AuthController.signup에 들어오는 것을 확인합니다.
- SignupService에서 normalizedEmail, 각 길이 검증, existsByEmail 결과를 확인합니다.
- UserMapper.xml의 INSERT가 password가 아니라 BCrypt 문자열을 저장하는지 확인합니다.
- 성공하면 /login?signup으로 이동하고 로그인 화면에 성공 안내가 표시됩니다.

### 11.2 로그인

- 로그인 폼이 /login으로 POST되지만 AuthController가 아니라 Spring Security가 처리한다는 점을 확인합니다.
- CustomUserDetailsService.loadUserByUsername에 이메일이 전달되는지 확인합니다.
- CustomUserDetails 안의 getUserId와 getNickname이 방 생성과 화면 표시에서 사용되는지 찾습니다.
- 성공 후 /rooms로 이동하고, Thymeleaf sec:authentication이 닉네임을 렌더링하는지 봅니다.

### 11.3 방 만들기

- rooms/create.html의 form field name이 RoomController의 @RequestParam 이름과 같은지 비교합니다.
- RoomService.createRoom에서 실패할 수 있는 조건을 일부러 입력해 roomError가 화면에 나타나는지 확인합니다.
- 성공 시 Room 객체에 생성된 roomId가 채워지고, 방장 room\_members INSERT가 뒤따르는지 확인합니다.
- redirect:/rooms/{roomId} 뒤 RoomController가 DB의 RoomList와 멤버 이름을 Model에 넣는지 확인합니다.

### 11.4 채팅과 준비 상태

- detail.html의 body data-\* 속성과 chat.js의 roomId, nickname, capacity를 비교합니다.
- 브라우저 Network에서 /ws 연결 후 CONNECTED, SUBSCRIBE, join SEND 순서를 확인합니다.
- 서버의 ChatController와 RoomPresenceController 중 어느 쪽이 메시지를 받는지 목적지로 구분합니다.
- 한 브라우저에서 채팅을 보내고, 다른 브라우저가 /topic/rooms/{id}/chat을 받아 렌더링하는지 확인합니다.

- 준비 버튼 클릭 후 RoomPresenceService의 Map이 바뀌고 전체 presence가 재방송되는지 확인합니다.

## 11.5 학습할 때 기록할 값

| 단계    | 관찰할 값                             | 왜 중요한가                      |
|-------|-----------------------------------|-----------------------------|
| HTTP  | URL, HTTP method, form field name | 어느 Controller 메서드로 들어가는지 결정 |
| 인증    | Principal, userId, nickname       | 화면과 WebSocket에서 현재 사용자를 식별  |
| DB    | roomId, user_id, current_players  | 영속 데이터와 화면 숫자의 관계 확인        |
| STOMP | destination, body, sessionId      | 채팅과 presence 처리 경로를 결정      |
| 화면    | Model 이름과 th:* 표현식                | 서버 값이 HTML에 어떻게 들어가는지 확인    |

## 12. 현재 코드에서 다음으로 배울 내용

현재 프로젝트는 웹 서비스의 뼈대를 학습하기에 좋은 상태입니다. 다음 단계에서는 화면을 더 만드는 것보다, 이미 있는 방과 참가자 개념을 실제 게임 상태로 확장하는 설계가 중요합니다.

### 12.1 기능 확장 순서

| 순서 | 학습 주제      | 구현할 핵심                                                     |
|----|------------|------------------------------------------------------------|
| 1  | 방 입장과 퇴장   | 비밀번호 검증, 정원 확인, room_members INSERT/DELETE, 중복 입장 방지       |
| 2  | 상태 모델      | RoomState, PlayerSession, GamePhase, Role, PlayerStatus 정의 |
| 3  | 게임 시작      | 방장 권한, 최소 인원, 전원 준비, WAITING -> 게임 페이즈 전이                  |
| 4  | 역할과 개인 메시지 | 마피아/경찰/의사/시민 배정과 개인 destination 권한                         |
| 5  | 투표와 승패     | 1인 1표, 동물 규칙, 동시 집계 보호, 승리 조건                              |
| 6  | 영속화와 운영    | 게임 기록, 전적, 재접속, 분산 환경, 통합 테스트                              |

### 12.2 현재 코드에서 특히 주의할 경계

- 방 입장과 WebSocket join은 다릅니다. 현재 GET /rooms/{roomId}는 화면을 열고, WebSocket join은 실시간 세션을 메모리에 등록합니다. DB 참가 등록까지 이어지지는 않습니다.
- DB 인원 수와 실시간 인원 수는 다를 수 있습니다. 로비 목록의 currentPlayers는 room\_members COUNT이고, 대기실의 숫자는 presence participants.length입니다.
- 채팅은 현재 방 참가 여부를 확인하지 않습니다. 로그인했고 방이 존재하면 메시지를 방송하므로, 실제 서비스에서는 세션의 방 참가 여부와 권한을 추가로 확인해야 합니다.
- fallback 화면은 실제 방을 만들지 않습니다. 존재하지 않는 roomId에 대한 예시 화면은 WebSocket과 일관되지 않으므로 운영 코드에서는 제거하거나 404 처리로 바꾸는 편이 좋습니다.

- **프로필은 아직 샘플 데이터입니다.**UserController가 userId를 받아도 항상 유진과 고정 전적을 Model에 넣습니다. UserMapper와 user\_stats를 연결하는 다음 과제가 필요합니다.

## 12.3 테스트를 시작하는 방법

현재 소스 트리에는 별도의 src/test 코드가 보이지 않습니다. 먼저 순수 규칙부터 테스트하는 것이 좋습니다. 예를 들어 SignupService의 이메일 정규화와 비밀번호 규칙, RoomService의 인원 범위와 해시 저장, RoomPresenceService의 join/ready/leave 상태를 단위 테스트로 분리할 수 있습니다. 그다음 Spring Security 테스트로 로그인과 보호 URL을 확인하고, 마지막으로 WebSocket 통합 테스트를 추가합니다.

### 추천 테스트 목록

#### 단위 테스트 후보

- 닉네임, 이메일, 비밀번호, 약관 검증
- 방 제목, 최대 인원, 비밀번호 검증
- 방 생성 시 game\_room과 host member의 순서
- presence join -> ready -> disconnect

#### 통합 테스트 후보

- 회원가입 -> 로그인 -> /rooms 접근
- 인증 사용자만 /rooms/new POST 가능
- /ws 연결 후 chat과 presence destination 처리

## 13. 핵심 용어 사전

| 용어                   | 이 프로젝트에서의 뜻                                                 |
|----------------------|-------------------------------------------------------------|
| Bean                 | Spring이 생성하고 관리하는 객체. Controller, Service, Config 등이 해당합니다. |
| Dependency Injection | 객체가 필요한 Service나 Mapper를 생성자 매개변수로 받는 방식입니다.                |
| Model                | Controller가 템플릿에 전달하는 이름과 값의 묶음입니다.                         |
| Thymeleaf            | HTML 안의 th:* 속성으로 서버 값을 화면에 넣는 템플릿 엔진입니다.                   |
| Domain               | 사용자나 방처럼 애플리케이션의 핵심 데이터를 표현하는 객체입니다.                        |
| DTO                  | 특정 요청이나 응답의 데이터 모양을 표현하는 객체입니다.                             |
| Mapper               | Java 메서드 호출과 SQL 실행을 연결하는 MyBatis 인터페이스입니다.                 |
| Transaction          | 여러 DB 작업을 하나의 성공 또는 실패 단위로 묶는 경계입니다.                        |
| Principal            | 현재 요청이나 WebSocket 세션의 로그인 사용자입니다.                           |
| WebSocket            | 브라우저와 서버가 연결을 유지한 채 양방향 통신하는 기술입니다.                         |
| STOMP                | 메시지를 destination과 frame 형식으로 주고받는 프로토콜입니다.                  |
| Broker               | topic을 구독한 여러 사용자에게 메시지를 전달하는 중계기입니다.                       |
| Presence             | 현재 연결된 참가자와 준비 상태 같은 실시간 접속 정보입니다.                          |
| BCrypt               | 비밀번호 원문 대신 검증 가능한 해시를 저장하는 방식입니다.                           |

## 마지막 복습 질문

- 회원가입 POST가 AuthController로 들어간 뒤 실제 INSERT는 어느 파일에서 실행되는가?
- 로그인 POST를 AuthController가 직접 처리하지 않는 이유는 무엇인가?
- RoomList.currentPlayers와 RoomPresenceState.participants의 차이는 무엇인가?
- 채팅의 /app 목적지와 /topic 목적지는 각각 어느 방향의 메시지인가?
- 준비 상태가 DB가 아니라 Map에 저장될 때 서버 재시작에서 어떤 일이 일어나는가?
- 실제 게임 시작 기능을 추가하려면 먼저 어떤 상태 모델을 만들어야 하는가?

이 질문에 파일 이름과 메서드 이름으로 답할 수 있으면, 현재 MAFIAGAME 코드의 큰 흐름은 이미 이해한 것입니다. 다음 학습에서는 기능을 추가할 때마다 먼저 '어떤 상태가 영속 데이터이고 어떤 상태가 실시간 메모리인가'를 구분해 보세요.